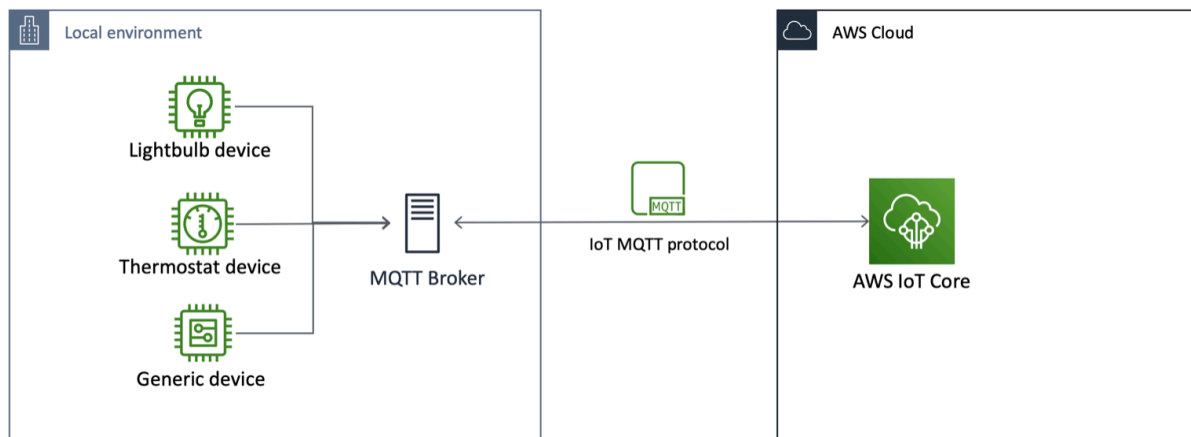


Tutorial 6 - Communicate to AWS IoT Core

Aim

This tutorial will show you how to implement the 'Bridge' capability to setup bi-directional exchange of data with AWS IoT Core through MQTT messages. This will enable your devices to communicate locally with the Mosquitto broker/other SDKs and with AWS IoT Core to benefit from the power of the AWS Cloud.



Configure the Bridge to AWS IoT

Create Your AWS Account

If you do not have an AWS account, please go to <https://aws.amazon.com> to create a **free personal root user account**. This course will not use any charge services, so the basic support plan is sufficient.

Sign up for AWS

Select a support plan

Choose a support plan for your business or personal account. [Compare plans and pricing examples](#)
[You can change your plan anytime in the AWS Management Console.](#)

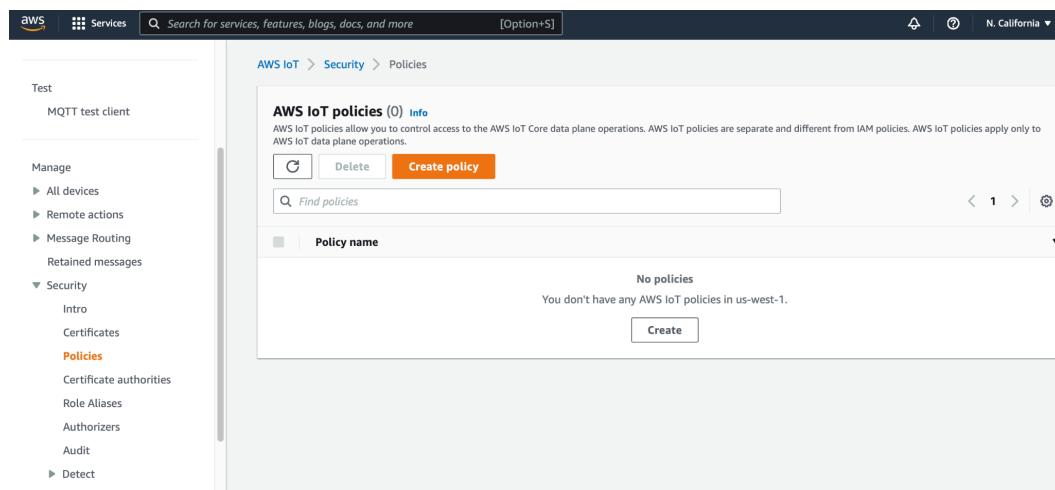
☒ Basic support - Free - Recommended for new users just getting started with AWS 24x7 self-service access to AWS resources - For account and billing issues only - Access to Personal Health Dashboard & Trusted Advisor 	☐ Developer support - From \$29/month - Recommended for developers experimenting with AWS - Email access to AWS Support during business hours 12 (business)-hour response times 	☐ Business support - From \$100/month - Recommended for running production workloads on AWS 24x7 tech support via email, phone, and chat 1-hour response times - Full set of Trusted Advisor best-practice recommendations 
--	--	--

Once you have received the activation email, your aws account is ready for use.

Create the resources on AWS IoT

Now we will need to configure the bridge so that we can create a bi-directional connection to AWS IoT Core. We will first use the AWS CLI to create the necessary resources on AWS IoT side.

1. Search **IoT core** on the top navigation bar, enter your AWS IoT page. Then, expand the **Security** option on the left side and click **Policies**, now you will be in the page showing below:



2. Now click **Create Policy**, enter the policy name (e.g. COMP6733MQTT). Under policy statement, set **policy effect** to **allow**, **policy action** and **policy resource** to *****.
(Note: Allowing all AWS IoT action (iot:*) is useful for testing but it's a best practice to increase security for a production setup)

Save the policy and now you can see your policy show on the page.

AWS IoT policies (1) [Info](#)

AWS IoT policies allow you to control access to the AWS IoT Core data plane operations. AWS IoT policies are separate and different from IAM policies. AWS IoT policies apply only to AWS IoT data plane operations.

< 1 > 

☐ **Policy name** ▼

☐ [COMP6733MQTT](#)

Policy effect	Policy action	Policy resource
Allow	*	*

3. Next step, we are going to create a Thing. Expand **All devices**, then go the **Things** page and create a new thing.

Manage

▼ All devices

Things

Thing groups

Thing types

Fleet metrics

4. Enter your thing name (e.g. COMP6733Thing), then keep other settings as default and go the next page: **Device certificate**. Please select the recommended one. In the next step, attach the policy that you just created (e.g. COMP6733MQTT) and click **Create thing**.

Device certificate

☒ **Auto-generate a new certificate (recommended)**

Generate a certificate, public key, and private key using AWS IoT's certificate authority.

Attach policies to certificate - *optional* [Info](#)

AWS IoT policies grant or deny access to AWS IoT resources. Attaching policies to the device certificate applies this access to the device.

Policies (1/1) [Refresh](#) [Create policy](#)

Select up to 10 policies to attach to this certificate.

< 1 > [Settings](#)

☒	Name
☒	COMP6733MQTT

[Cancel](#) [Previous](#) [Create thing](#)

5. After you creating your thing, AWS will allow you to download certificates and keys. **This is very important! This is the only time you can download these files.** Please download your **device certificate** xxx.crt and your **private and public key** files, as well as the **RootCA.pem** file. Now you should have 4 files in your local directory.

Download certificates and keys [Close](#)

Download certificate and key files to install on your device so that it can connect to AWS.

Device certificate

You can activate the certificate now, or later. The certificate must be active for a device to connect to AWS IoT.

Device certificate
0799cd214ac...te.pem.crt

[Deactivate certificate](#) [Download](#)

Key files

The key files are unique to this certificate and can't be downloaded after you leave this page. Download them now and save them in a secure place.

This is the only time you can download the key files for this certificate.

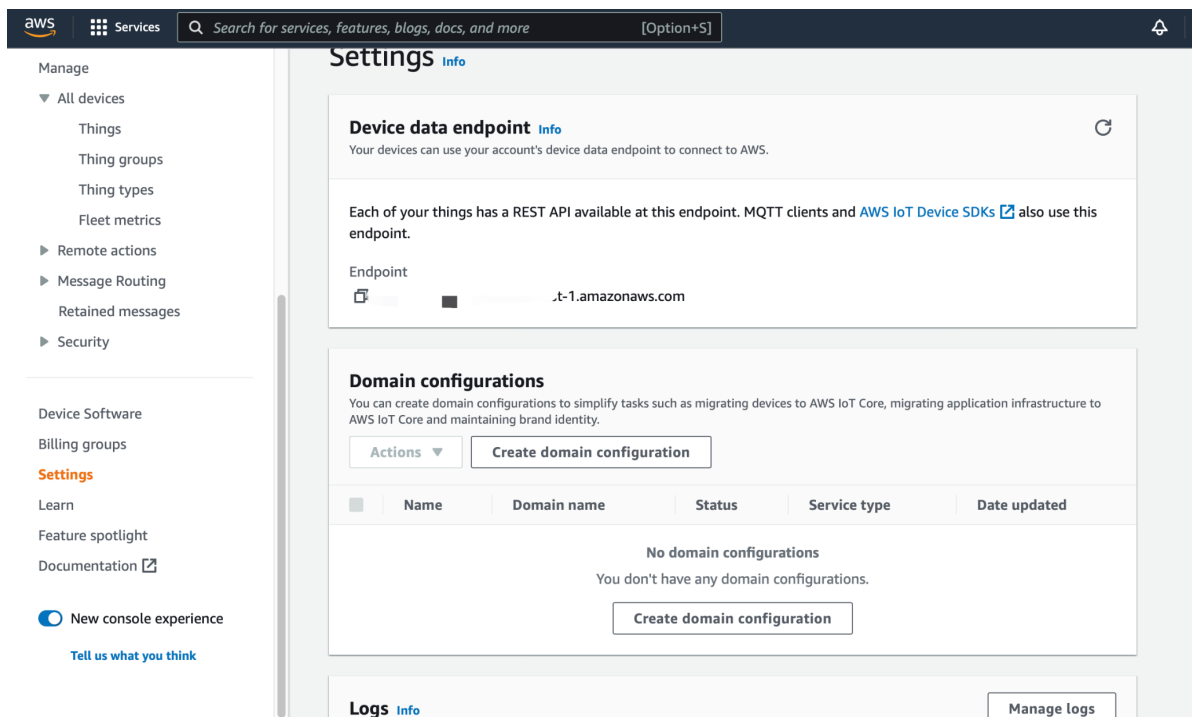
Public key file
0799cd214acdf532b539f4d...124b86d-public.pem.key

[Download](#)
Key downloaded

Private key file
0799cd214acdf532b539f4d...24b86d-private.pem.key

[Download](#)
Key downloaded

6. In the last step, navigate to the **Settings** page on AWS IoT. You can find your unique **Endpoint** (e.g. xxxxxx.amazonaws.com).



Sending Message to AWS IoT Core

With the files you obtained from previous steps, you are now ready to send the message to AWS IoT from your device.

You can configure the Mosquitto Bridge to AWS IoT with these files by looking at the tutorial: [Bridge Mosquitto MQTT Broker to AWS IoT](#) to publish/subscribe messages. However, we will introduce another method here, using an python sdk:

Using pip3 to install

`pip3 install AWSIoTPythonSDK`

Then, create a .py code under the same folder of your credential files, copy the following python3 code and change **endpoint address**, **credentials file name** to yours.

```
import time as t
import json
import AWSIoTPythonSDK.MQTTLib as AWSIoTPyMQTT

# Client configuration with endpoint and credentials
myClient = AWSIoTPyMQTT.AWSIoTMQTTClient("testDevice")
myClient.configureEndpoint('xxxx.amazonaws.com', 8883)
myClient.configureCredentials("AmazonRootCA1.pem", "0799cd214acdf532b539f4d0ecee9d00d1ac2be367d04939753e3bead124b86d-private.pem.key", "0799cd214acdf532b539f4d0ecee9d00d1ac2be367d04939753e3bead124b86d-
```

```
certificate.pem.crt")

myClient.connect()

for i in range(10):
    message = str(i+1)
    myClient.publish("test/comp6733",message,1)
    print("Published: '"+ message + "' to test/comp6733")
    t.sleep(0.5)

myClient.disconnect()
```

Go to your AWS IoT **MQTT test client** page, and subscribe the topic we used in the code: test/comp6733.

AWS IoT ×

Monitor

Connect

- Connect one device
- ▶ Connect many devices

Test

- MQTT test client**

MQTT test client [Info](#)

You can use the MQTT test client to monitor the MQTT messages being passed in your AWS account. Devices publish MQTT messages to inform devices and apps of changes and events. You can also publish MQTT messages to topics by using the MQTT test client.

Subscribe to a topic | **Publish to a topic**

Topic filter [Info](#)

The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

Enter the topic filter

Now run your python code, you should see 10 messages sending to your subscription topic as below:

Subscribe to a topic | **Publish to a topic**

Topic filter [Info](#)

The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

test/comp6733

▶ **Additional configuration**

Subscribe

Subscriptions	test/comp6733	Pause	Clear	Export	Edit
test/comp6733 ❤️ ✕	▼ test/comp6733 10				
	▼ test/comp6733 9				

Install Mosquitto MQTT Broker

Typically, you should install this on what you view as your local gateway which is the device that will be the link between your local devices and other local devices or to the AWS Cloud.

One way to install mosquitto into your own Linux machine is by typing the following command lines:

#update the ca-cert

```
sudo apt-get install --reinstall ca-certificates  
sudo update-ca-certificates
```

#Update the list of repositories with one containing the latest version of #Mosquitto and update the package lists

```
sudo apt-add-repository ppa:mosquitto-dev/mosquitto-ppa
```

```
sudo apt-get update
```

#Install the Mosquitto broker, Mosquitto clients

```
sudo apt-get install mosquitto
```

```
sudo apt-get install mosquitto-clients
```

Now you should be able to test the mosquitto broker locally.

In separate terminal windows do the following:

Start the broker on port 5000 (by default is 1883 but it might already in use):

```
mosquitto -p 5000
```

Start the command line subscriber:

```
mosquitto_sub -v -t 'test/topic' -p 5000
```

Publish test message with the command line publisher:

```
mosquitto_pub -t 'test/topic' -m 'helloCOMP6733' -p 5000
```

As well as seeing both the subscriber and publisher connection messages in the broker terminal the following should be printed in the subscriber terminal:

```
test/topic helloCOMP6733
```

You can also try on different OS such as Windows and MacOS.

Conclusion

This lesson introduce how to setup and configure your AWS account, and communicate via MQTT from your own device. For more information please refer to the related documentation.

Related Document

- [Bridge Mosquitto MQTT Broker to AWS IoT \(https://aws.amazon.com/blogs/iot/how-to-bridge-mosquitto-mqtt-broker-to-aws-iot/\)](https://aws.amazon.com/blogs/iot/how-to-bridge-mosquitto-mqtt-broker-to-aws-iot/)